



<useEffect>

[UNIFEI | Universidade Federal de Itajubá](#)

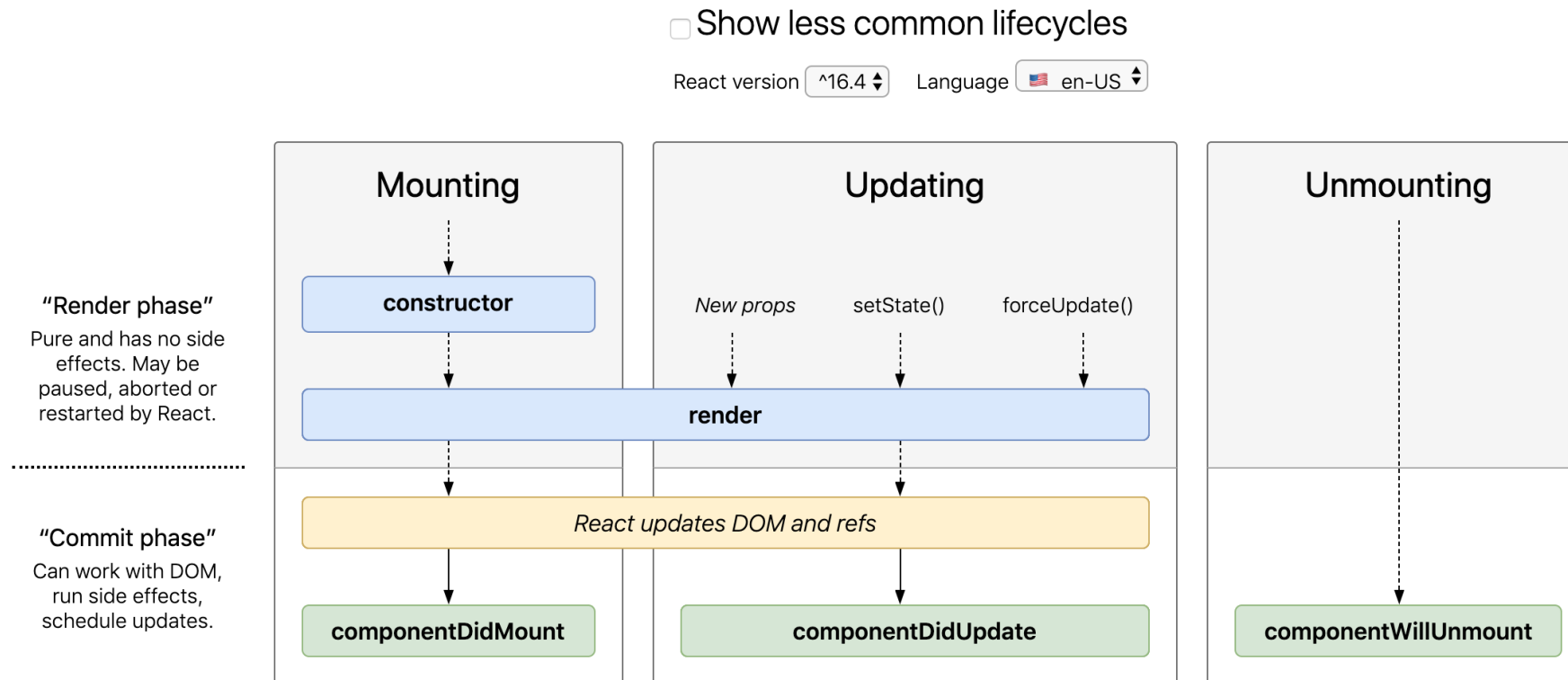
Instituto de Ciências Tecnológicas

ECO – <Dev Web e Dev Mobile>

eduardo.felipe@unifei.edu.br

Métodos de ciclo de vida (lifecycle methods)

- `componentDidMount()` – Usado em componentes de classe
- `useEffect()` – Usado em componentes funcionais



See project on [GitHub](#)

<https://projects.wojtekmaj.pl/react-lifecycle-methods-diagram/>

<https://pt-br.reactjs.org/docs/state-and-lifecycle.html#adding-lifecycle-methods-to-a-class>



Métodos de ciclo de vida - Funcionais

```
07: //comparacao com componentDidUpdate
08: useEffect( () => {
09:   console.log("componentDidUpdate");
10: });
11:
12: //comparacao com componentDidMount (array de dependencias para ser Mount)
13: useEffect( () => {
14:   console.log("componentDidMount");
15: }, []);
16:
17: //comparacao com componentDidMount e dependencias
18: useEffect(() => {
19:   console.log("componentDidMount com dependencia");
20:   console.log(counter);
21: }, [counter]);
```



Fetch()

- Para realizar requisições em um componente, é preciso ter atenção para não entrar em loop (race conditions).
- Usar um método de ciclo de vida adequado.
- Cuidado com as dependências.

```
const [dados, setDados] = useState([]);  
const url = "https://jsonplaceholder.typicode.com/users";
```

```
useEffect( () => {  
  console.log("Requisitando dados por um fetch")  
  fetch(url)  
    .then(resp => resp.json())  
    .then(resp => setDados(resp))  
}, [url]);
```



```
import React, { useState, useEffect } from "react";

export default function Usuarios() {
  const [usuarios, setUsuarios] = useState([]);
  const url = "https://jsonplaceholder.typicode.com/users";

  useEffect(() => {
    fetch(url)
      .then((resp) => resp.json())
      .then((resp) => setUsuarios(resp))
      .catch((err) => console.log(err));
  }, [url]);

  return (
    <div>
      {usuarios.map((item) => {
        return (
          <div key={item.id}>
            <span>{item.name} -- </span>
            <span>{item.email} </span>
          </div>
        );
      })}
    </div>
  );
}
```



Fetch() – Outro exemplo

```
const [dados, setDados] = useState([]);  
const url = "https://jsonplaceholder.typicode.com/users";
```

```
const getData = async (url) => {  
  console.log("Requisitando dados por um fetch", url);  
  const resp = await fetch(url);  
  const data = await resp.json();  
  setDados(data);  
}
```

```
useEffect( () => {  
  getData(url)  
}, [url]);
```



Quatro regras para os Hooks

- Os *Hooks* devem ser chamados em componentes funcionais;
- Devem ser criados no “topo” da cadeia de componentes;
- Pode-se criar vários *Hooks* para gerenciar estados ou “*effects*”;
- Deve-se observar a sequencia na chamada dos *Hooks* para obter sempre o mesmo resultado;
- Obs: **Não** se codifica a chamada de um Hook baseado em condicionais.



Referências

- <https://fbflipper.com>
- <https://jsonplaceholder.typicode.com/users>
- <https://reactnative.dev/docs/network>
- <https://pt-br.react.dev/reference/react/useEffect#reference>
- <https://reactnative.dev/docs/network#using-fetch>





•

eduardo.felipe@unifei.edu.br